



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SECJ3483 WEB TECHNOLOGY

GROUP LAB ASSIGNMENT - SECTION 01

Case Study: Secure Person BMI Full-Stack Web Application

Lecturer: Dr. Norizam Bin Katmon

Prepared by: Triple T

NO.	NAME	MATRICS NUMBER
1.	MUHAMMAD HARITH HAIKAL BIN MOHD SAID	A23CS0249
2.	MUHAMMAD FAKHRUL MUSTAQIM BIN SUHAIMI	A23CS0127
3.	MOHAMMAD IRFAN DANIAL BIN KEFLI	A23CS0113
4.	THAYAALLAN NAIDU A/L GANESAN	A23CS0281

Phase 1: Security Investigation

Test No	What I Tested	Expected Secure Behavior	Actual Result	Is It a Weakness?	Why?
1	Add BMI with invalid data (age: -5, weight: -70, height: 0)	Backend should reject invalid data	Backend accepted and saved the record (200 OK)	Yes	No backend validation exists
2	Access another user's BMI record (id=5) using user id=1 token	Backend should deny access	Backend returned the record with no check	Yes	No owner-based access control
3	Access staff route as normal user	Backend should deny access	Returned all BMI records (200 OK)	Yes	No role check on staff route
4	Access admin route as normal user	Backend should deny access	Returned all users including plain passwords	Yes	No role check on admin route
5	Submit XSS payload in notes ()	Payload should display as text	HTML was rendered, image tag executed	Yes	Frontend uses v-html on user input
6	Check API response fields	Password/hash should not be visible	API returns plain password and password_hash	Yes	SELECT * used, no field filtering
7	SQL Injection in login email (aiman@student.utm.my' --)	Input treated as data	Login succeeded with wrong password	Yes	String concatenation in SQL query
8	Update protected fields (user_id, bmi, category)	Backend should reject protected fields	All fields were updated (200 OK)	Yes	No field whitelist, mass assignment

Test 1:

POST ⌵ Send

Query Headers Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "name": "",
3   "age": -5,
4   "height": 0,
5   "weight": -70,
6   "bmi": 999,
7   "category": "Normal",
8   "user_id": 1,
9   "notes": "testing invalid BMI"
10 }
11
```

Status: 201 Created Size: 779 Bytes Time: 326 ms

Response Headers Cookies Results Docs </> ≡

```
1 {
2   "message": "BMI record created. This route trusts
3     frontend data.",
4   "person": {
5     "id": 21,
6     "user_id": 1,
7     "name": "",
8     "age": -5,
9     "height": "0.00",
10    "weight": "-70.00",
11    "bmi": "999.00",
12    "category": "Normal",
13    "notes": "testing invalid BMI",
14    "created_at": "2026-06-22 12:01:59"
15  },
16  "debug_received_body": {
17    "name": "",
18    "age": -5,
```

Response Chart

Test 2:

GET ⌵ Send

Query **Headers** Auth Body Tests Pre Run

HTTP Headers Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunde
☒	Authorization	Bearer eyJ1c2VyX2lkjoxLCJyb2xljoi
☐	header	value

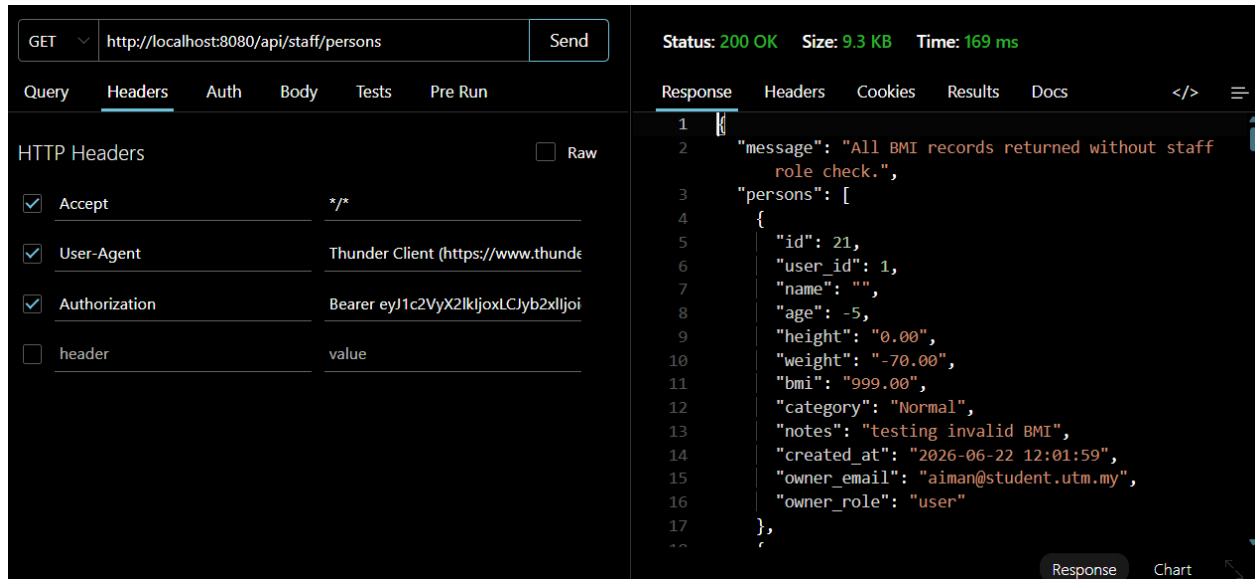
Status: 200 OK Size: 441 Bytes Time: 596 ms

Response Headers Cookies Results Docs </> ≡

```
1 {
2   "message": "Record returned without ownership check.",
3   "person": {
4     "id": 5,
5     "user_id": 5,
6     "name": "Muhammad Danish Azman",
7     "age": 22,
8     "height": "1.68",
9     "weight": "72.00",
10    "bmi": "25.51",
11    "category": "Overweight",
12    "notes": "Slightly above normal range",
13    "created_at": "2026-06-22 11:38:29"
14  },
15  "debug_sql": "SELECT * FROM persons WHERE id = 5"
16 }
```

Response Chart

Test 3:



Thunder Client interface for Test 3. The query is GET `http://localhost:8080/api/staff/persons`. The status is 200 OK, size is 9.3 KB, and time is 169 ms. The response is a JSON object with a message and a list of persons.

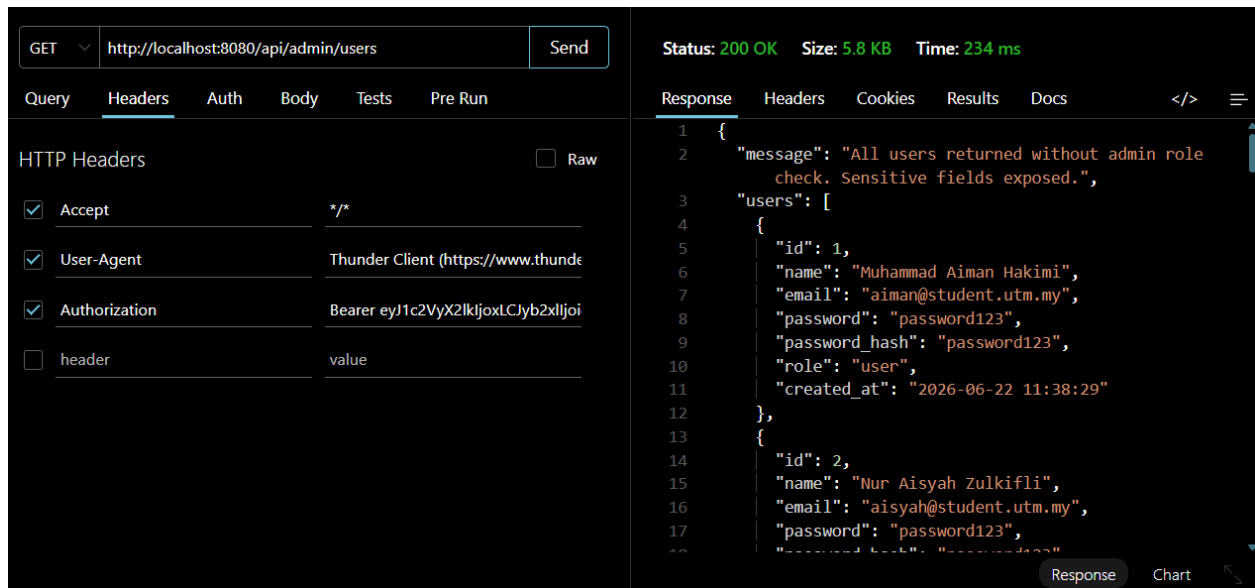
Query: GET `http://localhost:8080/api/staff/persons` **Send**

Headers: Accept `*/*`, User-Agent `Thunder Client (https://www.thunde`, Authorization `Bearer eyJ1c2VyX2lkjoxLCJyb2xljoi`

Response: Status: 200 OK, Size: 9.3 KB, Time: 169 ms. The response body is a JSON object with a message and a list of persons.

```
1 {
2   "message": "All BMI records returned without staff
3     role check.",
4   "persons": [
5     {
6       "id": 21,
7       "user_id": 1,
8       "name": "",
9       "age": -5,
10      "height": "0.00",
11      "weight": "-70.00",
12      "bmi": "999.00",
13      "category": "Normal",
14      "notes": "testing invalid BMI",
15      "created_at": "2026-06-22 12:01:59",
16      "owner_email": "aiman@student.utm.my",
17      "owner_role": "user"
18    },
19  ]
20 }
```

Test 4:



Thunder Client interface for Test 4. The query is GET `http://localhost:8080/api/admin/users`. The status is 200 OK, size is 5.8 KB, and time is 234 ms. The response is a JSON object with a message and a list of users.

Query: GET `http://localhost:8080/api/admin/users` **Send**

Headers: Accept `*/*`, User-Agent `Thunder Client (https://www.thunde`, Authorization `Bearer eyJ1c2VyX2lkjoxLCJyb2xljoi`

Response: Status: 200 OK, Size: 5.8 KB, Time: 234 ms. The response body is a JSON object with a message and a list of users.

```
1 {
2   "message": "All users returned without admin role
3     check. Sensitive fields exposed.",
4   "users": [
5     {
6       "id": 1,
7       "name": "Muhammad Aiman Hakimi",
8       "email": "aiman@student.utm.my",
9       "password": "password123",
10      "password_hash": "password123",
11      "role": "user",
12      "created_at": "2026-06-22 11:38:29"
13    },
14    {
15      "id": 2,
16      "name": "Nur Aisyah Zulkifli",
17      "email": "aisyah@student.utm.my",
18      "password": "password123",
19      "password_hash": "password123",
20      "role": "user",
21      "created_at": "2026-06-22 11:38:29"
22    }
23  ]
24 }
```

Test 5:

Person BMI Security Lab

Investigation GuideDashboardMy BMIAAdd BMILogout

My BMI Records

Investigation hint: Does the backend return only your records? What happens if you request another ID?

Load My RecordsTry Manual ID

BMI records returned. This route is intentionally weak.

Raw API response

```
{
  "ok": true,
  "status": 200,
  "data": {
    "message": "BMI records returned. This route is intentionally weak.",
    "persons": [
      {
        "id": 22,
        "user_id": 1,
        "name": "Test",
        "age": 21,
        "height": "1.70",
        "weight": "65.00",
        "bmi": "22.49",
        "category": "Normal",
        "notes": "<img src=x onerror=alert(1)>\n\n",
        "created_at": "2026-06-22 12:00:00"
      },
      {
        "id": 21,
        "user_id": 1,
        "name": "-",
        "age": -5,
        "height": "0.00",
        "weight": "-70.00",
        "bmi": "999.00",
        "category": "Normal",
        "notes": "testing invalid bmi",
        "created_at": "2026-06-22 12:01:59"
      },
      {
        "id": 1,
        "user_id": 1,

```

Test #22

Owner user_id: 1

Age: 21

Height: 1.70 m

Weight: 65.00 kg

BMI: 22.49

Category: Normal

Notes rendered with v-html intentionally:
 // i.e safer to use

DeleteTry ID + 1

#21

Owner user_id: 1

Age: -5

Height: 0.00 m

Weight: -70.00 kg

BMI: 999.00

Category: Normal

Notes rendered with v-html intentionally:
testing invalid BMI
// i.e safer to use
testing invalid BMI

DeleteTry ID + 1

Muhammad Aiman Hakimi #1

Owner user_id: 1

Age: 21

Height: 1.70 m

Weight: 65.00 kg

BMI: 22.49

Category: Normal

Notes rendered with v-html intentionally:
Student sample BMI record
// i.e safer to use
Student sample BMI record

DeleteTry ID + 1

Test 6:

GET http://localhost:8080/api/admin/users Send

Query Headers Auth Body Tests Pre Run

Query Parameters

☐ parameter value

Status: 200 OK Size: 5.8 KB Time: 245 ms

Response Headers Cookies Results Docs </>

```
1 {
2   "message": "All users returned without admin role
3   check. Sensitive fields exposed.",
4   "users": [
5     {
6       "id": 1,
7       "name": "Muhammad Aiman Hakimi",
8       "email": "aiman@student.utm.my",
9       "password": "password123",
10      "password_hash": "password123",
11      "role": "user",
12      "created_at": "2026-06-22 11:38:29"
13    },
14    {
15      "id": 2,
16      "name": "Nur Aisyah Zulkifli",
17      "email": "aisyah@student.utm.my",
18      "password": "password123",
19      "password_hash": "password123",
20      "role": "user",
21      "created_at": "2026-06-22 11:38:29"
22    }
23  ]
24 }
```

Response Chart

Test 7:

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/login`. The request body is a JSON object with `email` and `password` fields. The response is a 200 OK status with a JSON body containing user details and debug information.

Request:

```
POST http://localhost:8080/api/login
```

Request Body (JSON):

```
{
  "email": "aiman@student.utm.my" -- ",
  "password": "wrongpassword"
}
```

Response: Status: 200 OK, Size: 743 Bytes, Time: 78 ms

Response Body (JSON):

```
{
  "email": "aiman@student.utm.my",
  "password": "password123",
  "password_hash": "password123",
  "role": "user",
  "created_at": "2026-06-22 11:38:29",
  "debug_received_body": {
    "email": "aiman@student.utm.my" -- ",
    "password": "wrongpassword"
  },
  "debug_sql": "SELECT * FROM users WHERE email = 'aiman@student.utm.my' -- ' AND password = 'wrongpassword' LIMIT 1"
}
```

Test 8:

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/api/persons/1`. The request body is a JSON object with `user_id`, `bmi`, and `category` fields. The response is a 200 OK status with a JSON body containing a message, person details, and debug information.

Request:

```
PUT http://localhost:8080/api/persons/1
```

Request Body (JSON):

```
{
  "user_id": 2,
  "bmi": 10,
  "category": "Hacked"
}
```

Response: Status: 200 OK, Size: 600 Bytes, Time: 460 ms

Response Body (JSON):

```
{
  "message": "BMI record updated. This route allows unsafe field updates.",
  "person": {
    "id": 1,
    "user_id": 2,
    "name": "Muhammad Aiman Hakimi",
    "age": 21,
    "height": "1.70",
    "weight": "65.00",
    "bmi": "10.00",
    "category": "Hacked",
    "notes": "Student sample BMI record",
    "created_at": "2026-06-22 11:38:29"
  },
  "debug_received_body": {
    "user_id": 2,
    "bmi": 10,
    "category": "Hacked"
  }
}
```

Phase 2: Weakness Classification Table

Weakness Found	Frontend/Backend/API/Database	Security Category	Possible Risk
Negative weight accepted	Backend	Missing backend validation	Invalid data stored
User accesses another BMI record	API / Backend	Broken Object Level Authorization	Privacy violation
Normal user accesses staff route	API / Backend	Broken Function Level Authorization	Unauthorized data access
Normal user accesses admin route	API / Backend	Broken Function Level Authorization	Unauthorized admin action
XSS payload executes	Frontend	Cross-Site Scripting (XSS)	Browser script execution
API returns password_hash	API / Backend	Sensitive data exposure	Credential risk
SQL Injection affects login	Backend / Database	SQL Injection	Unauthorized data access
User changes user_id or bmi	API / Backend	Mass Assignment	Ownership manipulation

Phase 3: Problem-Solution Mapping

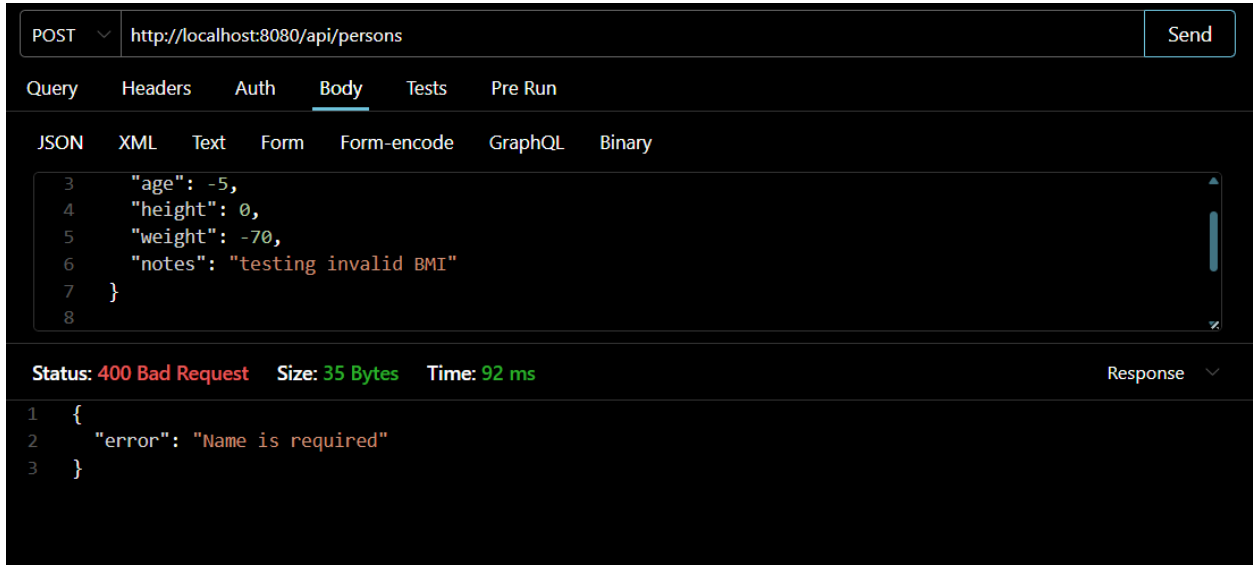
Problem	Root Cause	Where to Fix?	Proposed Solution	How to Retest?
Invalid BMI accepted	No backend validation	Backend	Validate name, age, height, weight	Submit invalid data again
User accesses other record	No ownership check	Backend API	Compare JWT user_id with record user_id	Access other user's record
User accesses staff route	No role check	Backend API	Check role from JWT — staff or admin only	Use user token on staff route
User accesses admin route	No role check	Backend API	Check role from JWT — admin only	Use user token on admin route
XSS executes	Unsafe v-html rendering	Frontend	Replace v-html with {{ }} interpolation	Submit XSS payload again
password_hash returned	SELECT * used	Backend API	Select only safe fields explicitly	Check JSON response
SQL Injection possible	String concatenation in SQL	Backend	Use prepared statements	Try injection input again
user_id can be changed	No field whitelist	Backend API	Allow only name, age, height, weight, notes	Send user_id in PUT request

Git Hub Commit Summary

Commit	Message	Changes
1	Initial commit - app running, security investigation completed	Deployed app, documented 8 security weaknesses
2	Add validation password hashing and prepared statements	Backend validation, bcrypt hashing, PDO prepared statements
3	Add JWT authentication and protected routes	firebase/php-jwt, signed tokens, all routes protected
4	Add RBAC ownership checks and final security fixes	Role checks, owner checks, XSS fix in BmiCard.vue

After Fix Testing

Test 1: Invalid BMI Data



Thunder Client interface showing a POST request to `http://localhost:8080/api/persons`. The request body is a JSON object with invalid values for age, height, and weight, and a note. The response is a 400 Bad Request with an error message.

Request:

- Method: POST
- URL: `http://localhost:8080/api/persons`
- Body (JSON):

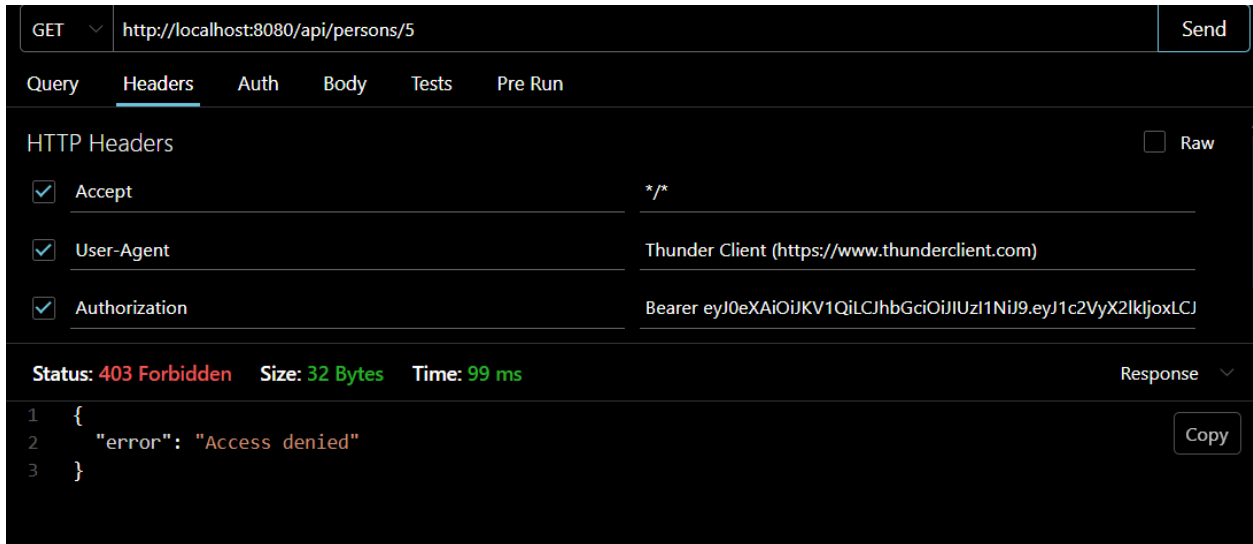
```
{  "age": -5,  "height": 0,  "weight": -70,  "notes": "testing invalid BMI"}
```

Response:

- Status: 400 Bad Request
- Size: 35 Bytes
- Time: 92 ms
- Response (JSON):

```
{  "error": "Name is required"}
```

Test 2: Access another user's BMI record



Thunder Client interface showing a GET request to `http://localhost:8080/api/persons/5`. The request headers include Accept, User-Agent, and Authorization. The response is a 403 Forbidden with an error message.

Request:

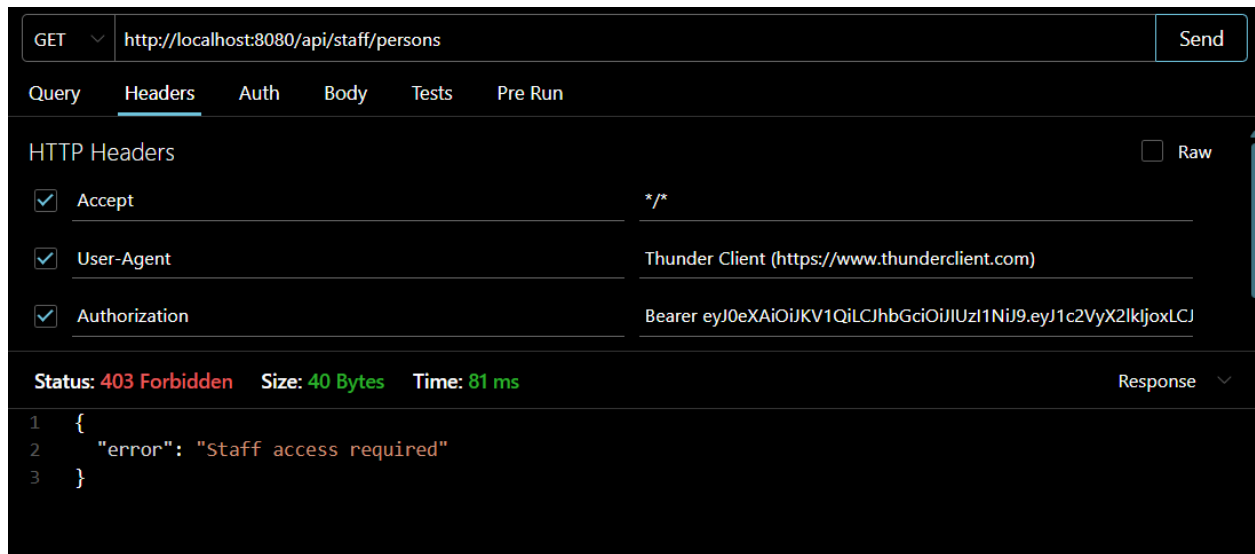
- Method: GET
- URL: `http://localhost:8080/api/persons/5`
- Headers:
 - Accept: */*
 - User-Agent: Thunder Client (<https://www.thunderclient.com>)
 - Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkjoxLCJ

Response:

- Status: 403 Forbidden
- Size: 32 Bytes
- Time: 99 ms
- Response (JSON):

```
{  "error": "Access denied"}
```

Test 3: Access staff route as normal user



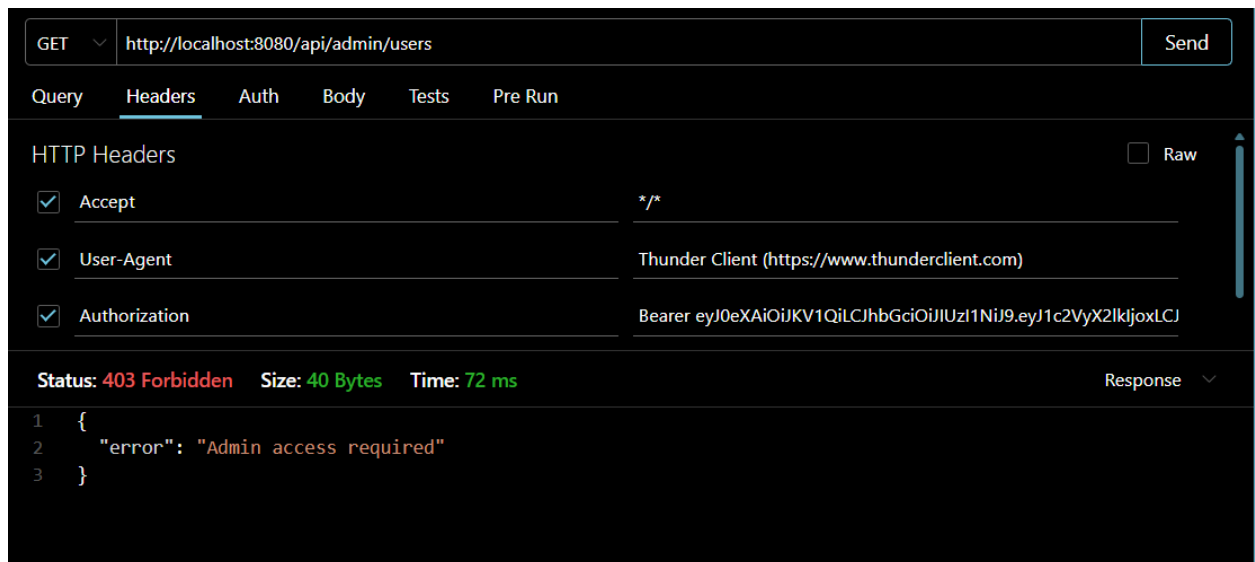
Thunder Client interface showing a GET request to `http://localhost:8080/api/staff/persons`. The request is sent with the following headers:

- Accept:** */*
- User-Agent:** Thunder Client (https://www.thunderclient.com)
- Authorization:** Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkjoxLCJ

The response status is **403 Forbidden**, with a size of **40 Bytes** and a time of **81 ms**. The response body is:

```
1 {
2   "error": "Staff access required"
3 }
```

Test 4: Access admin route as normal user



Thunder Client interface showing a GET request to `http://localhost:8080/api/admin/users`. The request is sent with the following headers:

- Accept:** */*
- User-Agent:** Thunder Client (https://www.thunderclient.com)
- Authorization:** Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkjoxLCJ

The response status is **403 Forbidden**, with a size of **40 Bytes** and a time of **72 ms**. The response body is:

```
1 {
2   "error": "Admin access required"
3 }
```

Test 5: XSS Payload in Notes

Notes

Frontend-calculated BMI

Frontend-calculated Category

► Insecure payload preview

Save BMI

BMI record created

Raw API response

```
{
  "ok": true,
  "status": 201,
  "data": {
    "message": "BMI record created",
    "person": {
      "id": 23,
      "user_id": 1,
      "name": "Rah",
      "age": 23,
      "height": "2.00",
      "weight": "80.00",
      "bmi": "20.00",
      "category": "Normal",
      "notes": "<img src=x onerror=alert(1)>",
      "created_at": "2026-06-26 04:52:00"
    }
  }
}
```

Rah #23
Owner user_id: 1
Age: 23
Height: 2.00 m
Weight: 80.00 kg
BMI: 20.00
Category: Normal
Notes:

Delete

Try ID + 1

Test 6: Login response has no password_hashe

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/login`. The request body is a JSON object with email and password fields. The response status is 200 OK, and the response body is a JSON object containing a success message, a token, and user details.

```
POST http://localhost:8080/api/login
```

Query Headers Auth **Body** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "email": "aiman@student.utm.my",
3   "password": "password123"
4 }
```

Status: 200 OK Size: 401 Bytes Time: 285 ms Response

```
1 {
2   "message": "Login successful",
3   "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxCjYb2x1IjoidXNlciIsImVtYWlsIjoiyWltYW5Ac3R1ZGVudC51dG0ubXkiLCJpYXQiOjE3ODI0MjAwNzIsImV4cCI6MTc4MjQyMzY3Mn0.R2TSb0kUwshDyi65oKs4a2aU17phTpldWSWFACa_Qmg",
4   "user": {
5     "id": 1,
6     "name": "Muhammad Aiman Hakimi",
7     "email": "aiman@student.utm.my".
8   }
9 }
```

Response Chart

Test 7: SQL Injection

The screenshot shows a REST client interface with a POST request to `http://localhost:8080/api/login`. The request body is a JSON object where the email field contains a SQL injection payload. The response status is 401 Unauthorized, and the response body is a JSON object containing an error message.

```
POST http://localhost:8080/api/login
```

Query Headers Auth **Body** Tests Pre Run

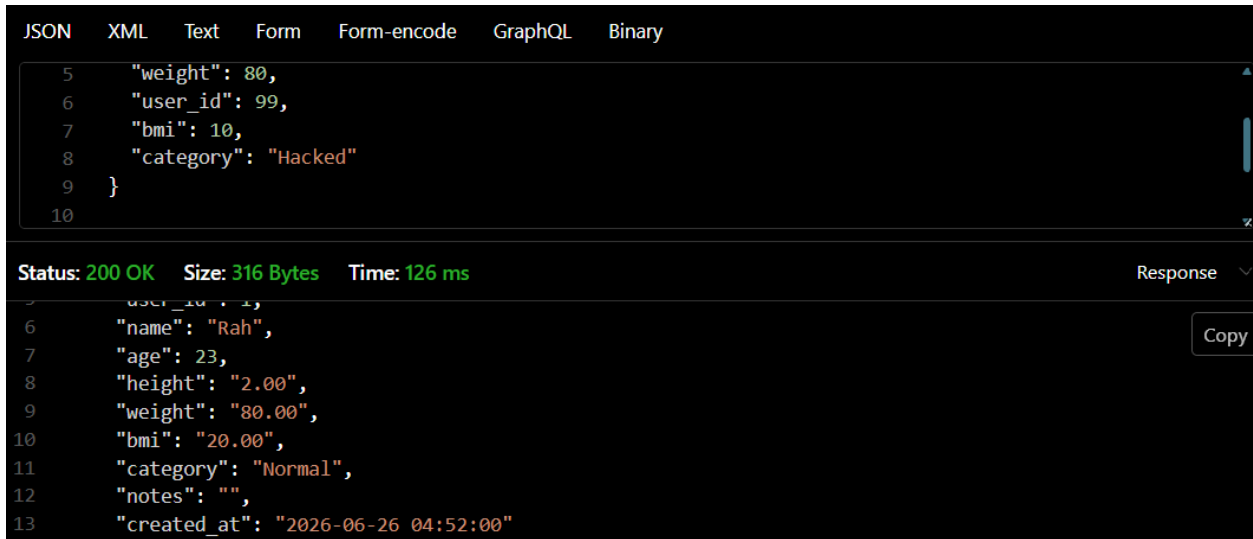
JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "email": "aiman@student.utm.my' -- ",
3   "password": "wrongpassword"
4 }
5
```

Status: 401 Unauthorized Size: 44 Bytes Time: 135 ms

```
1 {
2   "error": "Invalid email or password"
3 }
```

Test 8: Update protected fields



Security Testing Results

Test	Before Fix	After Fix
1. Invalid BMI	200 OK, accepted	400 Bad Request
2. Access other user's record	200 OK, returned data	403 Access denied
3. Staff route as normal user	200 OK, returned all BMI	403 Staff access required
4. Admin route as normal user	200 OK, exposed passwords	403 Admin access required
5. XSS in notes	Alert executed	Displayed as plain text
6. API exposes password_hash	password + hash visible	Only id, name, email, role
7. SQL Injection login	Login succeeded	401 Invalid email or password
8. Update protected fields	user_id/bmi/category changed	Ignored, backend recalculates

Reflection

36. Which weakness was easiest to find? Why?

The API response exposing password_hash (Test 6) was the easiest to find. It only required checking the login or admin response — the sensitive field was immediately visible in the JSON output without any special technique.

37. Which weakness was most dangerous? Why?

SQL Injection (Test 7) was the most dangerous. It allowed login bypass using a crafted email input, which could grant unauthorized access to any account without knowing the password. This could compromise the entire user database.

38. Why is frontend validation not enough?

Frontend validation can be bypassed using tools like Postman or Thunder Client to send requests directly to the API. The backend must always validate input independently because it cannot trust data sent from the browser.

39. Why should BMI calculation be performed at the backend?

A user can manipulate frontend JavaScript to send any BMI value they want. If the backend trusts the frontend's BMI value, false health records can be stored. The backend must recalculate BMI from height and weight to ensure data integrity.

40. What is the difference between authentication and authorization?

Authentication verifies who the user is (login with email and password returns a JWT). Authorization decides what the user is allowed to do (checking role and ownership before allowing access to a route).

41. What is the difference between RBAC and owner-based access control?

RBAC (Role-Based Access Control) grants access based on the user's role — for example, only staff or admin can view all BMI records. Owner-based access control grants access based on who owns the data — for example, a user can only update or delete their own BMI record.

42. Why is JWT alone not enough to protect all data?

JWT only proves that the user is logged in. It does not automatically check whether the user has the right role or owns the resource being accessed. Role checks and ownership checks must be implemented separately in each route.

43. Why should API responses exclude password_hash?

If password_hash is returned in the API response, an attacker who intercepts the response can attempt offline brute-force attacks to recover the original password. The frontend never needs the password hash, so it should never be included in any response.

44. Why is Vue route guard not enough to protect backend routes?

Vue route guards only control navigation in the browser. An attacker can bypass the frontend entirely and send requests directly to the backend API using Postman. The backend must enforce its own access control independently.

45. How did you prove that your security fix works?

We tested each fix using Thunder Client before and after applying the code changes. Before the fix, the insecure backend accepted invalid data, exposed passwords, and allowed unauthorized access. After the fix, the same requests returned 400, 401, or 403 responses with appropriate error messages, confirming each vulnerability was resolved.